

Paso 1: Leer el CSV de Titanic desde S3 (Extracción y Validación)

En esta etapa se conecta la función Lambda con Amazon S3 utilizando la biblioteca boto3. Antes de procesar cualquier información, se ejecuta una validación en dos niveles utilizando bloques de control de excepciones (try-except): primero se verifica mediante head_bucket que el bucket destino exista y sea accesible, y posteriormente se comprueba con get_object la existencia real del archivo titanic.csv. Si las validaciones son exitosas, los bytes del archivo se transforman en memoria a un DataFrame de Pandas mediante BytesIO, asegurando la integridad del pipeline desde el origen.

```
new_data.py X
processing > new_data.py
1 import boto3
2 import pandas as pd
3 from io import BytesIO
4 from botocore.exceptions import ClientError
5
6 s3 = boto3.client("s3")
7
8 # CONSTANTES DE TU ENTORNO
9 BUCKET = "xideralaws-curso-enrique2026"
10 KEY = "titanic.csv"
11
12 def get_data():
13     # 1. Validar si el Bucket existe y tienes acceso
14     try:
15         s3.head_bucket(Bucket=BUCKET)
16     except ClientError as e:
17         error_code = e.response['Error']['Code']
18         if error_code == '404':
19             raise Exception(f"Error: El bucket '{BUCKET}' no existe.")
20         elif error_code == '403':
21             raise Exception(f"Error: No tienes permisos para acceder al bucket '{BUCKET}'.")
22         else:
23             raise Exception(f"Error al validar el bucket: {str(e)}")
24
25     # 2. Validar si el archivo CSV existe y leerlo
26     try:
27         response = s3.get_object(Bucket=BUCKET, Key=KEY)
28         df = pd.read_csv(BytesIO(response["Body"].read()))
29         return df
30     except ClientError as e:
31         error_code = e.response['Error']['Code']
32         if error_code == 'NoSuchKey':
33             raise Exception(f"Error: El archivo '{KEY}' no existe en el bucket '{BUCKET}'.")
34         else:
35             raise Exception(f"Error al leer el archivo CSV: {str(e)}")
```

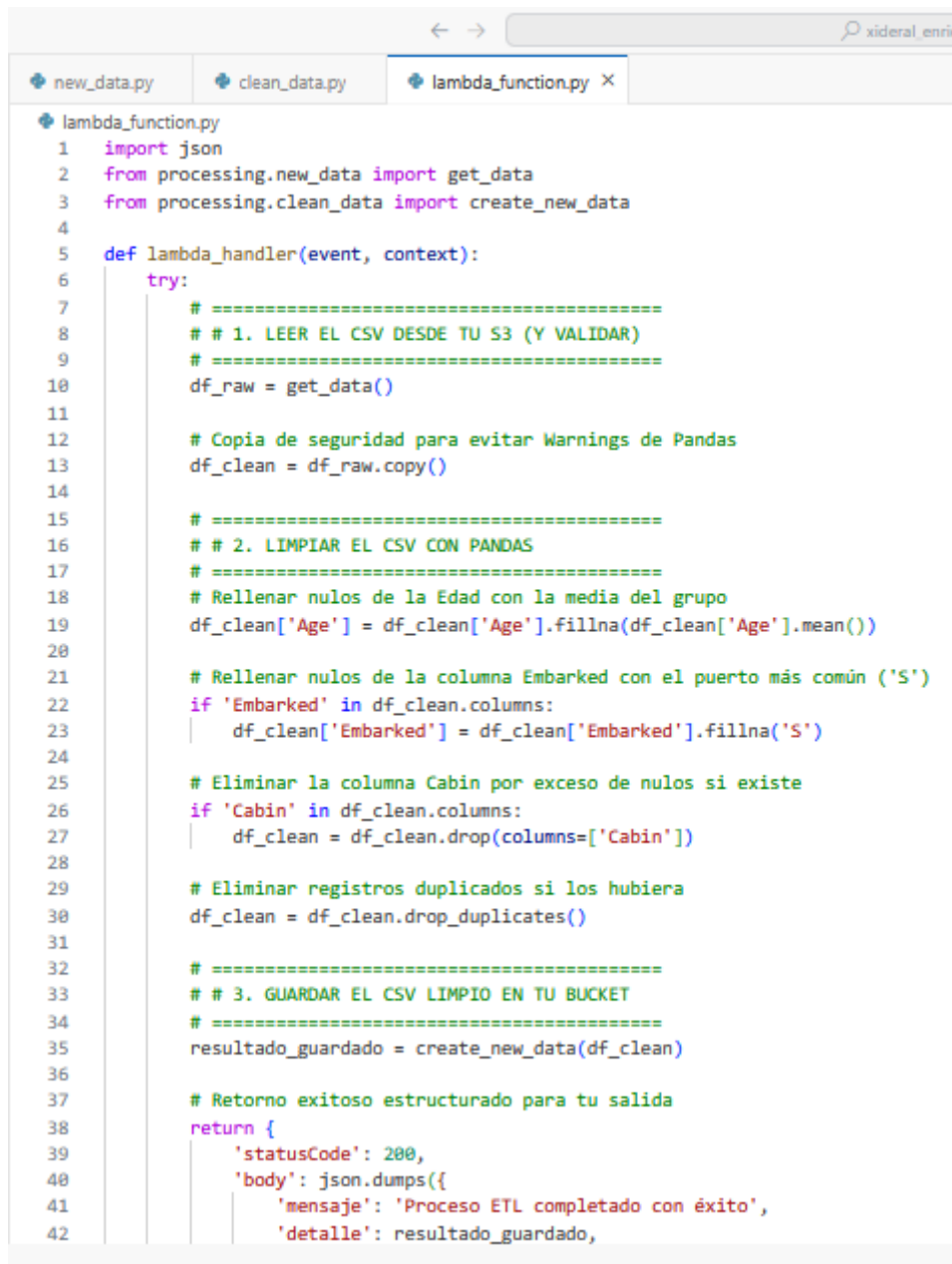
Paso 2: Limpiar el CSV (Transformación de Datos)

Una vez cargado el dataset en memoria, se realiza una copia explícita del DataFrame para mitigar riesgos de fragmentación o advertencias de asignación en Pandas (SettingWithCopyWarning). La etapa de transformación aplica reglas de negocio específicas para el tratamiento de la información: se gestionan los valores nulos (NaN) de la columna 'Age' imputándoles la media matemática del conjunto; se estandarizan las filas vacías de la variable 'Embarked' asignándoles el puerto más frecuente ('S'); y se descartan columnas con alto índice de ausencia de datos o irrelevantes para el modelado, como 'Cabin'. Finalmente, se eliminan registros duplicados para garantizar la unicidad de los datos.

```
new_data.py × clean_data.py ×
processing > clean_data.py
1  import boto3
2  from io import StringIO
3
4  s3 = boto3.client("s3")
5
6  # CONSTANTES DE TU ENTORNO (Mismo bucket, destino diferente)
7  BUCKET = "xideralaws-curso-enrique2026"
8  KEY_DESTINO = "titanic_clean.csv"
9
10 def create_new_data(df_clean):
11     try:
12         # Convertimos el DataFrame limpio a una cadena CSV en memoria
13         csv_buffer = StringIO()
14         df_clean.to_csv(csv_buffer, index=False)
15
16         # Subimos el CSV nuevo a tu bucket
17         s3.put_object(
18             Bucket=BUCKET,
19             Key=KEY_DESTINO,
20             Body=csv_buffer.getvalue(),
21             ContentType="text/csv"
22         )
23         return f"Archivo limpio guardado exitosamente como '{KEY_DESTINO}'."
24     except Exception as e:
25         raise Exception(f"Error al guardar el archivo limpio en S3: {str(e)}")
```

Guardar el CSV limpio en el mismo Bucket (Carga / Load)

En la fase final del proceso ETL, el DataFrame transformado y depurado se convierte nuevamente a texto plano en formato CSV. Para optimizar el rendimiento y evitar el uso de almacenamiento local en el entorno de ejecución de la Lambda, se utiliza un búfer de texto en memoria (StringIO). Utilizando el método `put_object` de `boto3`, el flujo de datos se sube directamente de regreso a AWS S3 bajo el nombre de `titanic_clean.csv`, alojándose en el mismo bucket de trabajo original y concluyendo el ciclo de persistencia de datos limpios.



```
1 import json
2 from processing.new_data import get_data
3 from processing.clean_data import create_new_data
4
5 def lambda_handler(event, context):
6     try:
7         # =====
8         # # 1. LEER EL CSV DESDE TU S3 (Y VALIDAR)
9         # =====
10        df_raw = get_data()
11
12        # Copia de seguridad para evitar Warnings de Pandas
13        df_clean = df_raw.copy()
14
15        # =====
16        # # 2. LIMPIAR EL CSV CON PANDAS
17        # =====
18        # Rellenar nulos de la Edad con la media del grupo
19        df_clean['Age'] = df_clean['Age'].fillna(df_clean['Age'].mean())
20
21        # Rellenar nulos de la columna Embarked con el puerto más común ('S')
22        if 'Embarked' in df_clean.columns:
23            df_clean['Embarked'] = df_clean['Embarked'].fillna('S')
24
25        # Eliminar la columna Cabin por exceso de nulos si existe
26        if 'Cabin' in df_clean.columns:
27            df_clean = df_clean.drop(columns=['Cabin'])
28
29        # Eliminar registros duplicados si los hubiera
30        df_clean = df_clean.drop_duplicates()
31
32        # =====
33        # # 3. GUARDAR EL CSV LIMPIO EN TU BUCKET
34        # =====
35        resultado_guardado = create_new_data(df_clean)
36
37        # Retorno exitoso estructurado para tu salida
38        return {
39            'statusCode': 200,
40            'body': json.dumps({
41                'mensaje': 'Proceso ETL completado con éxito',
42                'detalle': resultado_guardado,
```

```
new_data.py clean_data.py lambda_function.py X
lambda_function.py
5 def lambda_handler(event, context):
6     try:
17         # =====
18         # Rellenar nulos de la Edad con la media del grupo
19         df_clean['Age'] = df_clean['Age'].fillna(df_clean['Age'].mean())
20
21         # Rellenar nulos de la columna Embarked con el puerto más común ('S')
22         if 'Embarked' in df_clean.columns:
23             df_clean['Embarked'] = df_clean['Embarked'].fillna('S')
24
25         # Eliminar la columna Cabin por exceso de nulos si existe
26         if 'Cabin' in df_clean.columns:
27             df_clean = df_clean.drop(columns=['Cabin'])
28
29         # Eliminar registros duplicados si los hubiera
30         df_clean = df_clean.drop_duplicates()
31
32         # =====
33         # # 3. GUARDAR EL CSV LIMPIO EN TU BUCKET
34         # =====
35         resultado_guardado = create_new_data(df_clean)
36
37         # Retorno exitoso estructurado para tu salida
38         return {
39             'statusCode': 200,
40             'body': json.dumps({
41                 'mensaje': 'Proceso ETL completado con éxito',
42                 'detalle': resultado_guardado,
43                 'filas_originales': len(df_raw),
44                 'filas_procesadas': len(df_clean)
45             }, default=str)
46         }
47
48     except Exception as e:
49         # Si el bucket o el archivo no existen, caerá aquí y te avisará el motivo real
50         return {
51             'statusCode': 400,
52             'body': json.dumps({
53                 'mensaje': 'Error durante la ejecución del pipeline',
54                 'motivo': str(e)
55             })
56         }
```

IDE interface showing a Lambda function deployment and execution results. The function is named `lambda_function.py` and is deployed to the `current` environment. The execution results show a successful status with a response body containing ETL process details.

```
lambda_function.py
1 import json
2 from processing.new_data import get_data
3 from processing.clean_data import create_new_data
4
5 def lambda_handler(event, context):
6     try:
7         # =====
8         # # 1. LEER EL CSV DESDE TU S3 (Y VALIDAR)
9         # =====
10        df_raw = get_data()
11
12        # Copia de seguridad para evitar warnings de Pandas
13        df_clean = df_raw.copy()
14
15        # =====
16        # # 2. LIMPIAR EL CSV CON PANDAS
```

Execution Results:

```
Status: Succeeded
Test Event Name: test

Response:
{
  "statusCode": 200,
  "body": "{\"mensaje\": \"Proceso ETL completado con \\u0092xito!\", \"detalle\": \"Archivo limpio guardado exitosamente como 'titanic_clean.csv'.\", \"filas_originales\": 891, \"filas_procesadas\": 891}\""
}
```

Function Logs:

```
START RequestId: dca4e4da-4265-4197-b0b3-ba74578Febb0 Version: $LATEST
END RequestId: dca4e4da-4265-4197-b0b3-ba74578Febb0
REPORT RequestId: dca4e4da-4265-4197-b0b3-ba74578Febb0 Duration: 4989.62 ms Billed Duration: 7874 ms Memory Size: 128 MB Max Memory Used: 128 MB Init Duration: 2884.00 ms
Request ID: dca4e4da-4265-4197-b0b3-ba74578Febb0
```

AWS Management Console interface showing the Amazon S3 bucket `xideralaws-curso-enrique2026`. The bucket contains three objects: `hola.txt`, `titanic_clean.csv`, and `titanic.csv`.

Amazon S3 Buckets

xideralaws-curso-enrique2026

Objetos (3)

Nombre	Tipo	Última modificación	Tamaño	Clase de almacenamiento
hola.txt	txt	10 Jul 2026 5:16:14 PM CST	17.0 B	Estándar
titanic_clean.csv	csv	10 Jul 2026 5:39:33 PM CST	61.9 KB	Estándar
titanic.csv	csv	10 Jul 2026 2:51:37 AM CST	58.9 KB	Estándar